Case Study Work

Johan Nino Espino

2026-10-24

```
# 1. Load the original data
train_raw <- read.csv("data/data-train.csv")
test_raw  <- read.csv("data/data-test.csv")

train <- train_raw
test  <- test_raw

predictors <- c("St", "Re", "Fr")
raw_moments <- paste0("R_moment_", 1:4)
```

```
# 2. Check required columns
stopifnot(
  all(c(predictors, raw_moments) %in% names(train)),
  all(predictors %in% names(test))
)

# Check that the required columns are numeric and have no missing values.
# Fr = Inf is an intentional simulation setting, not a missing value.
stopifnot(
  all(vapply(train[c(predictors, raw_moments)], is.numeric, logical(1))),
  all(vapply(test[predictors], is.numeric, logical(1))),
  !anyNA(train[c(predictors, raw_moments)]),
  !anyNA(test[predictors])
)

# Re, St, and the raw moments must be finite.
stopifnot(
  all(is.finite(as.matrix(train[c("St", "Re", raw_moments)]))),
  all(is.finite(as.matrix(test[c("St", "Re")]))),
  all(train$Fr > 0),
```

```
  all(test$Fr > 0)
)
```

```
# 3. Convert raw moments into distribution summaries
m1 <- train$R_moment_1
m2 <- train$R_moment_2
m3 <- train$R_moment_3
m4 <- train$R_moment_4

train$mean_volume <- m1
train$variance_volume <- m2 - m1^2

# Stop and investigate if any variance is nonpositive.
if (any(train$variance_volume <= 0)) {
  stop("At least one calculated variance is nonpositive. Inspect those rows.")
}

train$sd_volume <- sqrt(train$variance_volume)

central_moment_3 <- m3 - 3 * m1 * m2 + 2 * m1^3
central_moment_4 <- m4 - 4 * m1 * m3 + 6 * m1^2 * m2 - 3 * m1^4

train$skewness_volume <- central_moment_3 / train$sd_volume^3
train$kurtosis_volume <- central_moment_4 / train$sd_volume^4

response_names <- c(
  "mean_volume", "variance_volume", "sd_volume",
  "skewness_volume", "kurtosis_volume"
)

stopifnot(
  all(is.finite(as.matrix(train[response_names])))
)
```

```
# 4. Print a compact preparation check
preparation_check <- data.frame(
  dataset = c("Training", "Test"),
  rows = c(nrow(train_raw), nrow(test_raw)),
  original_columns = c(ncol(train_raw), ncol(test_raw)),
  missing_entries = c(sum(is.na(train_raw)), sum(is.na(test_raw))),
  duplicate_input_settings = c(
    sum(duplicated(train_raw[predictors])),
```

```
    sum(duplicated(test_raw[predictors]))
  ),
  infinite_Fr_settings = c(
    sum(is.infinite(train_raw$Fr)),
    sum(is.infinite(test_raw$Fr))
  )
)

knitr::kable(preparation_check, caption = "Data preparation checks")
```

Table 1: Data preparation checks

dataset	rows	original_columns	missing_entries	duplicate_input_settings	infinite_Fr_settings
Training	89	7	0	0	30
Test	23	3	0	0	12

```
# Preview the predictors and calculated responses
knitr::kable(
  head(train[c(predictors, response_names)]),
  digits = 5,
  caption = "First six training observations"
)
```

Table 2: First six training observations

St	Re	Fr	mean_volume	variance_volume	sd_volume	skewness_volume	kurtosis_volume
0.10	224	0.052	0.00216	0.13035	0.36103	305.42800	93371.6555
3.00	224	0.052	0.00379	0.47041	0.68586	216.76225	47012.2515
0.70	224	Inf	0.00291	0.04349	0.20854	90.58974	8216.7778
0.05	90	Inf	0.06353	0.08662	0.29431	17.67980	420.2523
0.70	398	Inf	0.00037	0.00622	0.07889	257.58554	66372.7845
2.00	90	0.300	0.14780	1.98496	1.40888	12.64607	165.0999

```
# Summarize the calculated responses
summary(train[response_names])
```

```
  mean_volume       variance_volume       sd_volume         skewness_volume
 Min.   :0.000222   Min.   :1.000e-04   Min.   : 0.01006   Min.   : 11.97
 1st Qu.:0.002157   1st Qu.:2.447e-02   1st Qu.: 0.15643   1st Qu.: 72.55
```

```
 Median :0.002958   Median :8.078e-02   Median : 0.28423   Median :110.12
 Mean   :0.040394   Mean   :9.249e+01   Mean   : 3.65112   Mean   :162.81
 3rd Qu.:0.087868   3rd Qu.:5.268e-01   3rd Qu.: 0.72579   3rd Qu.:269.54
 Max.   :0.172340   Max.   :1.044e+03   Max.   :32.31526   Max.   :344.91
 kurtosis_volume
 Min.   :   150.5
 1st Qu.:  5622.3
 Median : 12158.7
 Mean   : 39749.6
 3rd Qu.: 72732.4
 Max.   :132136.7
```

```r
#| label: predictor-setup
#| message: false

library(ggplot2)

# Check that all test predictor levels occur in the training data
stopifnot(
  all(test$Re %in% train$Re),
  all(test$Fr %in% train$Fr),
  all(test$St %in% train$St)
)

# Use identical factor levels in training and test data
Re_levels <- sort(unique(train$Re))
Fr_levels <- sort(unique(train$Fr))
St_levels <- sort(unique(train$St))

train$Re_factor <- factor(train$Re, levels = Re_levels)
test$Re_factor  <- factor(test$Re, levels = Re_levels)

train$Fr_factor <- factor(train$Fr, levels = Fr_levels)
test$Fr_factor  <- factor(test$Fr, levels = Fr_levels)

# Inverse Froude number: Inf becomes 0
train$inv_Fr <- 1 / train$Fr
test$inv_Fr  <- 1 / test$Fr

# Compact summary of the original predictors
predictor_summary <- data.frame(
  Predictor = c("Re", "Fr", "St"),
```

```
  Training_levels = sapply(train[predictors], function(x) length(unique(x))),
  Test_levels = sapply(test[predictors], function(x) length(unique(x))),
  Training_min = sapply(train[predictors], min),
  Training_max = sapply(train[predictors], max),
  row.names = NULL
)

# Explicitly match the displayed predictor order
predictor_summary <- data.frame(
  Predictor = predictors,
  Training_levels = sapply(train[predictors], function(x) length(unique(x))),
  Test_levels = sapply(test[predictors], function(x) length(unique(x))),
  Training_min = sapply(train[predictors], min),
  Training_max = sapply(train[predictors], max),
  row.names = NULL
)

knitr::kable(
  predictor_summary,
  caption = "Predictor ranges and number of distinct settings"
)
```

Table 3: Predictor ranges and number of distinct settings

Predictor	Training_levels	Test_levels	Training_min	Training_max
St	14	13	0.050	3
Re	3	3	90.000	398
Fr	3	3	0.052	Inf

```
knitr::kable(
  data.frame(Fr = Fr_levels, Inverse_Fr = 1 / Fr_levels),
  digits = 4,
  caption = "Froude number and its reciprocal"
)
```

Table 4: Froude number and its reciprocal

Fr	Inverse_Fr
0.052	19.2308
0.300	3.3333

Fr	Inverse_Fr
Inf	0.0000

```
#| label: predictor-counts
#| fig-width: 7
#| fig-height: 3.5

# Combine predictors only; no response values are used here
design_data <- rbind(
  data.frame(train[predictors], Dataset = "Training"),
  data.frame(test[predictors], Dataset = "Test")
)

design_data$Dataset <- factor(
  design_data$Dataset,
  levels = c("Training", "Test")
)

# Helper function for count plots
plot_setting_counts <- function(variable, setting_levels, axis_label) {

  plot_data <- data.frame(
    Setting = factor(design_data[[variable]], levels = setting_levels),
    Dataset = design_data$Dataset
  )

  ggplot(plot_data, aes(x = Setting, fill = Dataset)) +
    geom_bar(position = "dodge") +
    scale_fill_manual(values = c("Training" = "#2166AC", "Test" = "#D6604D")) +
    labs(
      x = axis_label,
      y = "Number of simulations",
      fill = "Dataset"
    ) +
    theme_bw(base_size = 11) +
    theme(legend.position = "top")
}

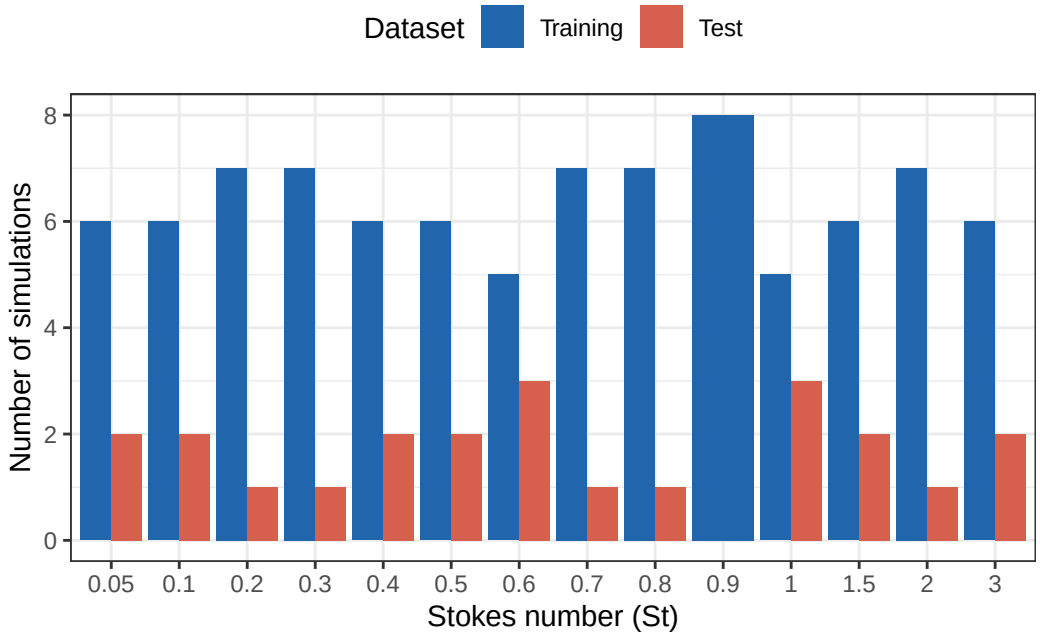
plot_setting_counts("Re", Re_levels, "Reynolds number (Re)")
```




```
plot_setting_counts("Fr", Fr_levels, "Froude number (Fr)")
```



```
plot_setting_counts("St", St_levels, "Stokes number (St)")
```




```
#| label: simulation-counts

# Include every combination, even combinations with no observations
design_counts <- expand.grid(
  Re = Re_levels,
  Fr = Fr_levels
)

design_counts <- design_counts[
  order(design_counts$Re, design_counts$Fr),
]

design_counts$Training <- mapply(
  function(re, fr) sum(train$Re == re & train$Fr == fr),
  design_counts$Re,
  design_counts$Fr
)

design_counts$Test <- mapply(
  function(re, fr) sum(test$Re == re & test$Fr == fr),
```

```
  design_counts$Re,
  design_counts$Fr
)

design_counts$Total <- design_counts$Training + design_counts$Test

knitr::kable(
  design_counts,
  row.names = FALSE,
  caption = "Simulation counts within each Re and Fr combination"
)
```

Table 5: Simulation counts within each Re and Fr combination

Re	Fr	Training	Test	Total
90	0.052	12	2	14
90	0.300	10	4	14
90	Inf	9	5	14
224	0.052	14	0	14
224	0.300	13	1	14
224	Inf	12	2	14
398	0.052	10	4	14
398	0.300	0	0	0
398	Inf	9	5	14

```
# Check for exact input settings shared by training and test data
shared_settings <- merge(
  unique(train[predictors]),
  unique(test[predictors]),
  by = predictors
)

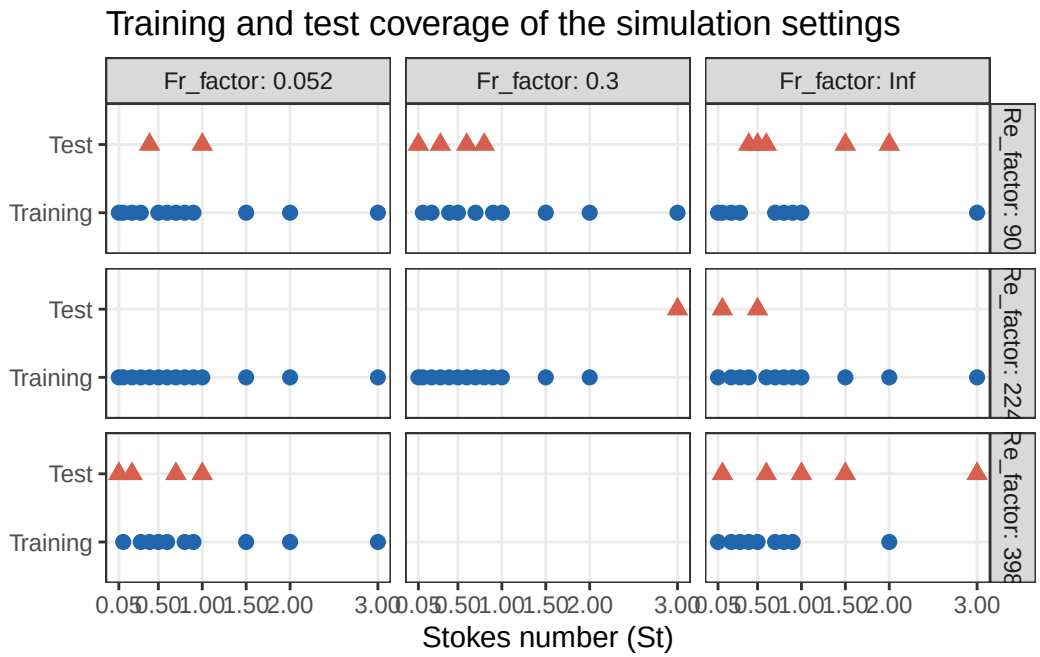
cat(
  "\nNumber of exact input settings shared by training and test:",
  nrow(shared_settings),
  "\n"
)
```

```
Number of exact input settings shared by training and test: 0
```

```r
#| label: simulation-coverage
#| fig-width: 8
#| fig-height: 6

design_data$Re_factor <- factor(design_data$Re, levels = Re_levels)
design_data$Fr_factor <- factor(design_data$Fr, levels = Fr_levels)

ggplot(
  design_data,
  aes(x = St, y = Dataset, color = Dataset, shape = Dataset)
) +
  geom_point(size = 2.5) +
  facet_grid(
    Re_factor ~ Fr_factor,
    labeller = label_both,
    drop = FALSE
  ) +
  scale_color_manual(
    values = c("Training" = "#2166AC", "Test" = "#D6604D")
  ) +
  scale_shape_manual(
    values = c("Training" = 16, "Test" = 17)
  ) +
  scale_x_continuous(
    breaks = c(0.05, 0.5, 1, 1.5, 2, 3)
  ) +
  labs(
    title = "Training and test coverage of the simulation settings",
    x = "Stokes number (St)",
    y = NULL
  ) +
  theme_bw(base_size = 11) +
  theme(
    legend.position = "none",
    panel.grid.minor = element_blank()
  )
```




```
#| label: response-plotting-data

response_columns <- c(
  "mean_volume",
  "sd_volume",
  "skewness_volume",
  "kurtosis_volume"
)

response_labels <- c(
  "Mean",
  "Standard deviation",
  "Skewness",
  "Kurtosis"
)

# Confirm that the responses exist and are finite
stopifnot(
  all(response_columns %in% names(train)),
  all(is.finite(as.matrix(train[response_columns]))))
)

# One row per simulation and response
```

```r
response_long <- do.call(
  rbind,
  lapply(seq_along(response_columns), function(j) {
    data.frame(
      St = train$St,
      Re_factor = train$Re_factor,
      Fr_factor = train$Fr_factor,
      Response = response_labels[j],
      Value = train[[response_columns[j]]]
    )
  })
)

response_long$Response <- factor(
  response_long$Response,
  levels = response_labels
)

# The log-scale plots below require positive values.
# Stop rather than silently removing nonpositive observations.
stopifnot(all(response_long$Value > 0))

# Consistent colors for Reynolds-number settings
Re_colors <- c(
  "90" = "#2166AC",
  "224" = "#1B7837",
  "398" = "#B2182B"
)
```

```r
#| label: response-distributions
#| fig-width: 8
#| fig-height: 5.5

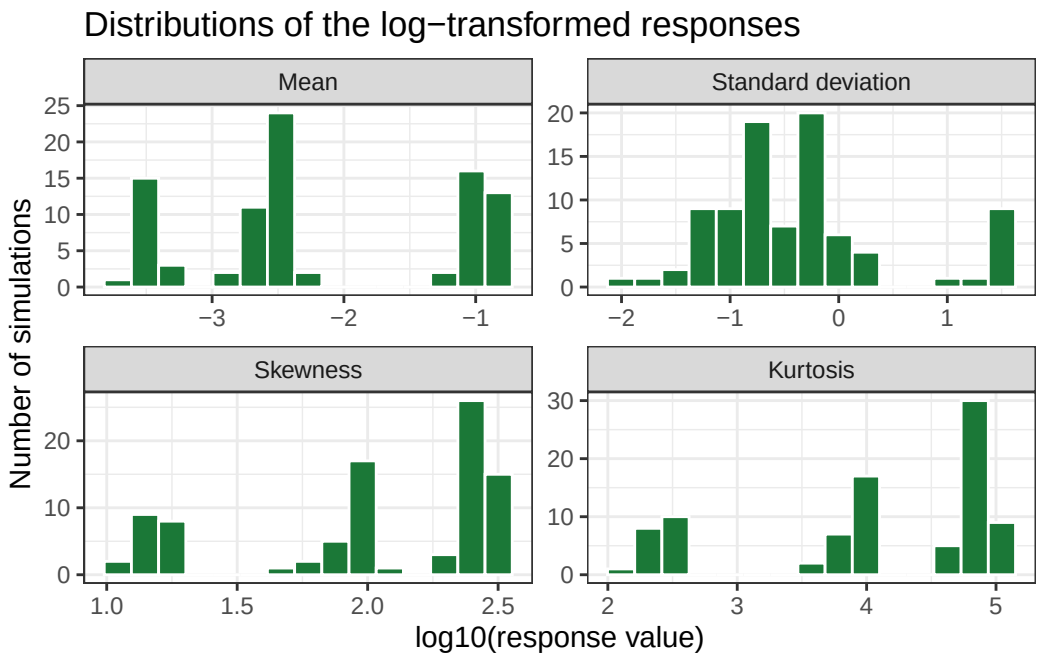
# Original-scale distributions
ggplot(response_long, aes(x = Value)) +
  geom_histogram(
    bins = 15,
    fill = "#2166AC",
    color = "white"
  ) +
  facet_wrap(
    ~ Response,
```

```
    scales = "free",
    ncol = 2
  ) +
  labs(
    title = "Distributions of the training responses",
    x = "Response value",
    y = "Number of simulations"
  ) +
  theme_bw(base_size = 11)
```




```
# Distributions after taking log10
ggplot(response_long, aes(x = log10(Value))) +
  geom_histogram(
    bins = 15,
    fill = "#1B7837",
    color = "white"
  ) +
  facet_wrap(
    ~ Response,
    scales = "free",
    ncol = 2
  ) +
```

```
  labs(
    title = "Distributions of the log-transformed responses",
    x = "log10(response value)",
    y = "Number of simulations"
  ) +
  theme_bw(base_size = 11)
```




```
#| label: responses-versus-St
#| fig-width: 8
#| fig-height: 3.8

for (response_name in response_labels) {

  plot_data <- response_long[
    response_long$Response == response_name,
  ]

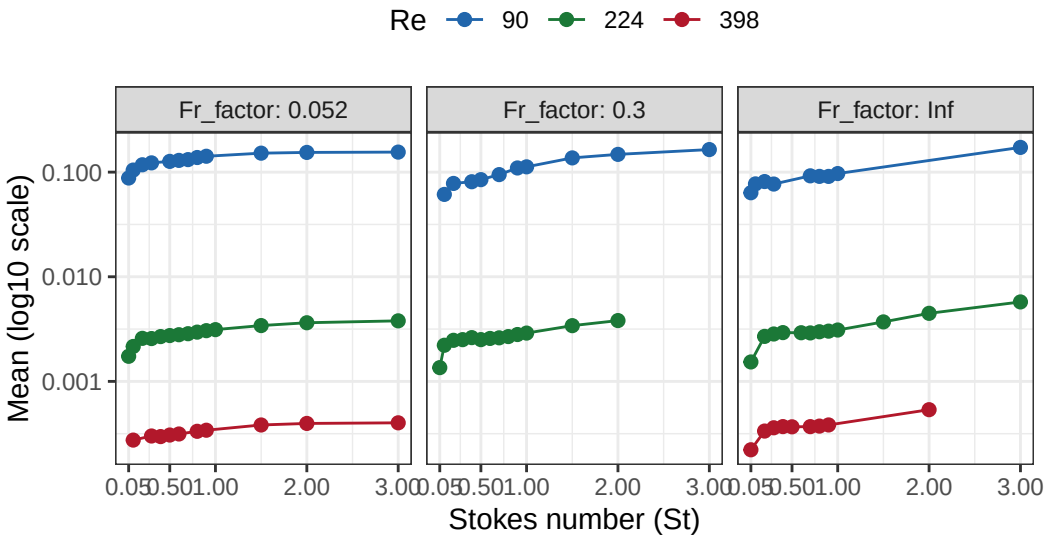
  p <- ggplot(
    plot_data,
    aes(
      x = St,
      y = Value,
```

```
      color = Re_factor,
      group = Re_factor
    )
  ) +
    geom_line(linewidth = 0.5) +
    geom_point(size = 2) +
    facet_wrap(
      ~ Fr_factor,
      nrow = 1,
      labeller = label_both,
      drop = FALSE
    ) +
    scale_color_manual(values = Re_colors) +
    scale_y_log10() +
    scale_x_continuous(
      breaks = c(0.05, 0.5, 1, 2, 3)
    ) +
    labs(
      title = paste(response_name, "versus Stokes number"),
      subtitle = "Lines connect available training observations within each Re-Fr regime",
      x = "Stokes number (St)",
      y = paste(response_name, "(log10 scale)"),
      color = "Re"
    ) +
    theme_bw(base_size = 11) +
    theme(legend.position = "top")

  print(p)
}
```

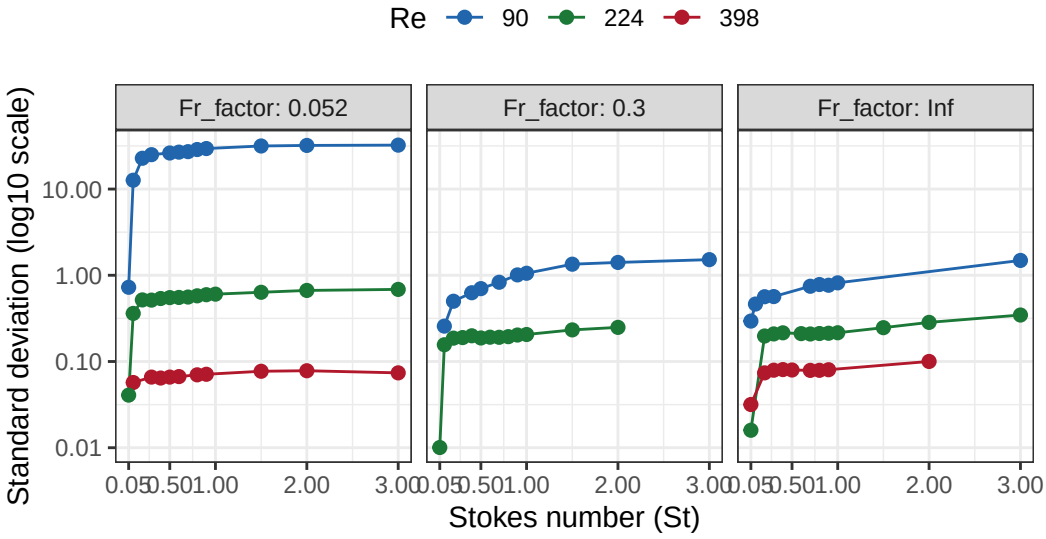
Mean versus Stokes number

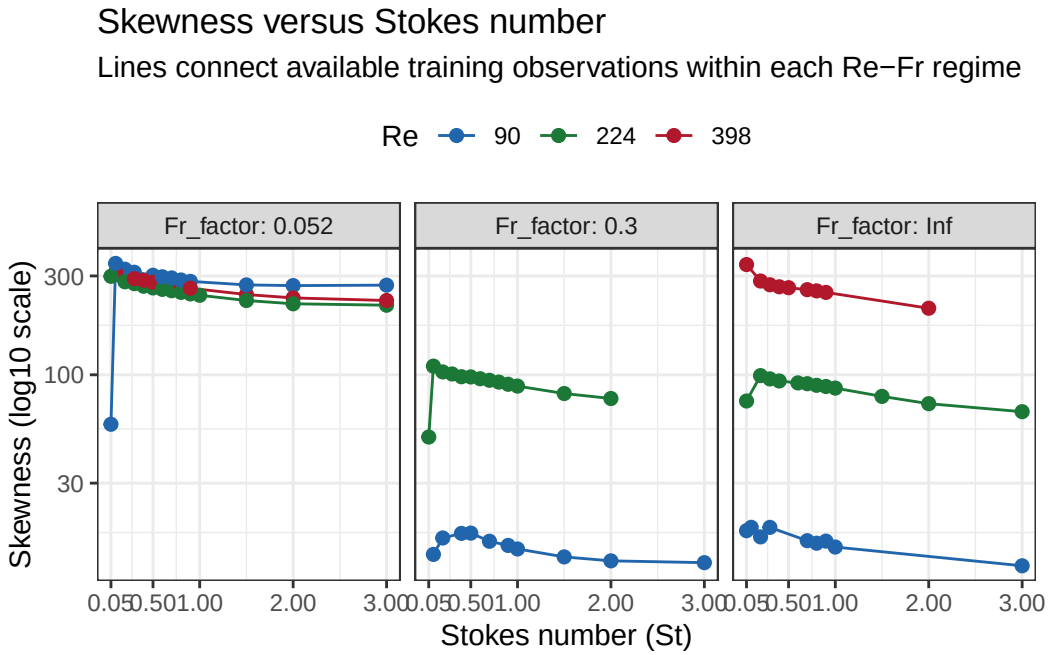
Lines connect available training observations within each Re−Fr regime

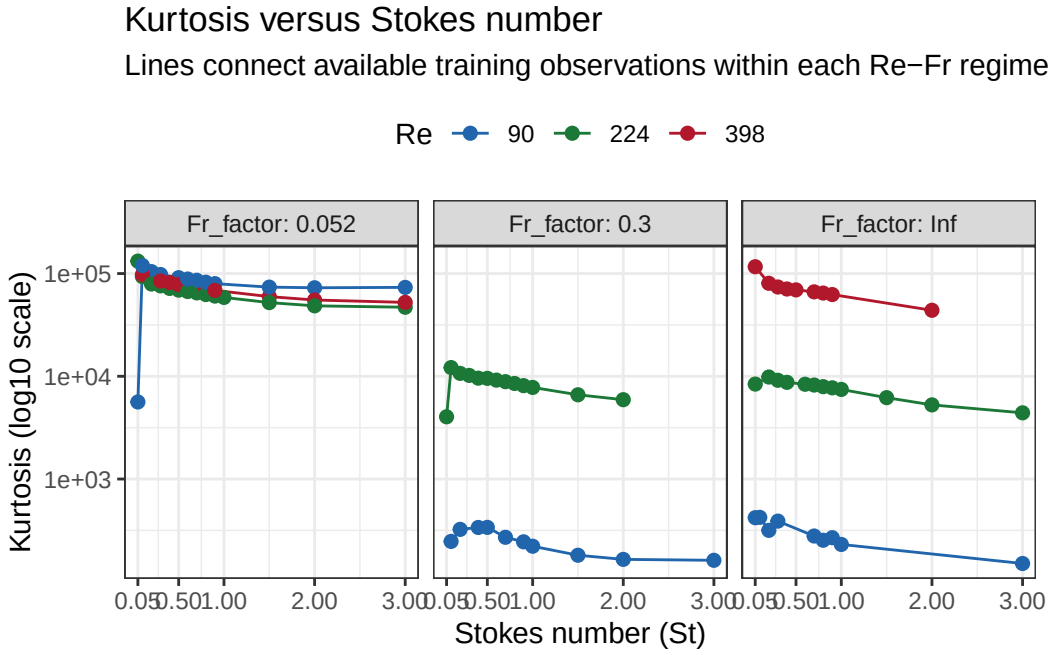


Standard deviation versus Stokes number

Lines connect available training observations within each Re−Fr regime

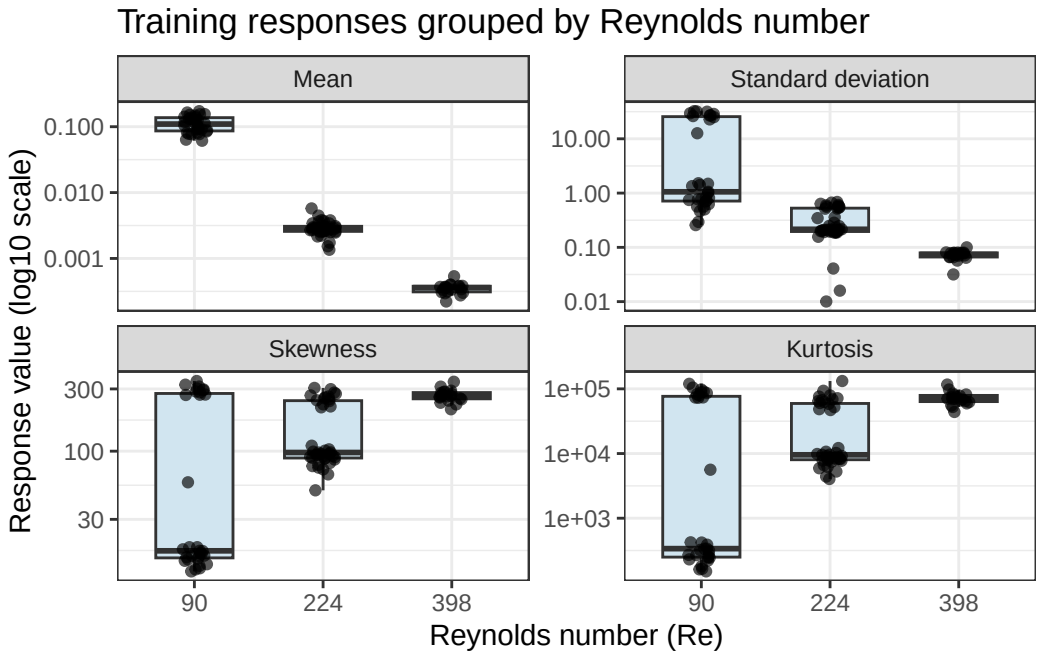






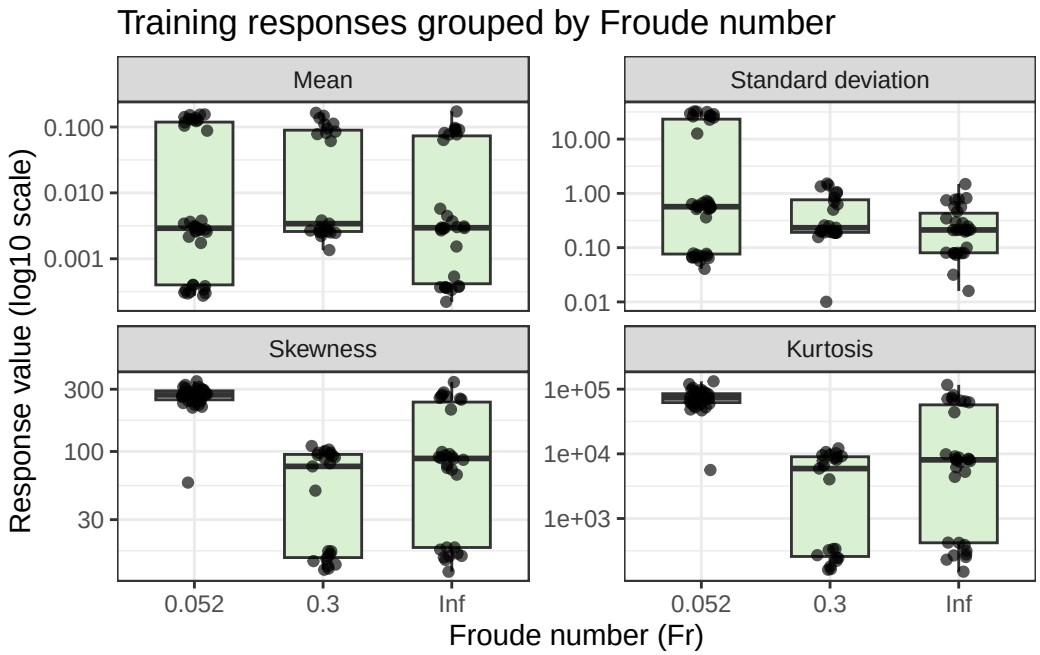

```
#| label: responses-by-Re-and-Fr
#| fig-width: 8
#| fig-height: 6
```

```r
# Responses grouped by Reynolds number
ggplot(
  response_long,
  aes(x = Re_factor, y = Value)
) +
  geom_boxplot(
    fill = "#D1E5F0",
    outlier.shape = NA,
    width = 0.6
  ) +
  geom_point(
    position = position_jitter(
      width = 0.10,
      height = 0,
      seed = 325
    ),
    alpha = 0.65,
    size = 1.5
  ) +
  scale_y_log10() +
  facet_wrap(~ Response, scales = "free_y", ncol = 2) +
  labs(
    title = "Training responses grouped by Reynolds number",
    x = "Reynolds number (Re)",
    y = "Response value (log10 scale)"
  ) +
  theme_bw(base_size = 11)
```




```
# Responses grouped by Froude number
ggplot(
  response_long,
  aes(x = Fr_factor, y = Value)
) +
  geom_boxplot(
    fill = "#D9F0D3",
    outlier.shape = NA,
    width = 0.6
  ) +
  geom_point(
    position = position_jitter(
      width = 0.10,
      height = 0,
      seed = 325
    ),
    alpha = 0.65,
    size = 1.5
  ) +
  scale_y_log10() +
  facet_wrap(~ Response, scales = "free_y", ncol = 2) +
  labs(
    title = "Training responses grouped by Froude number",
```

```
    x = "Froude number (Fr)",
    y = "Response value (log10 scale)"
  ) +
  theme_bw(base_size = 11)
```




```
#| label: candidate-transformations

# Check domains before taking logarithms
stopifnot(
  all(train$St > 0), all(test$St > 0),
  all(train$Re > 0), all(test$Re > 0),
  all(as.matrix(train[response_columns]) > 0)
)

# Candidate predictor transformations
train$log10_St <- log10(train$St)
test$log10_St  <- log10(test$St)

train$log10_Re <- log10(train$Re)
test$log10_Re  <- log10(test$Re)

# Candidate response transformations: training data only
```

```r
for (response in response_columns) {
  train[[paste0("log10_", response)]] <- log10(train[[response]])
}

# Predictor transformation reference
transformation_reference <- data.frame(
  Predictor = c("St", "Re", "Fr"),
  Candidate = c("log10(St)", "log10(Re)", "1/Fr"),
  Purpose = c(
    "Represent proportional changes in particle inertia",
    "Represent proportional changes in Reynolds number",
    "Represent relative gravity strength; Inf becomes zero"
  )
)

knitr::kable(
  transformation_reference,
  caption = "Candidate predictor transformations"
)
```

Table 6: Candidate predictor transformations

Predictor	Candidate	Purpose
St	log10(St)	Represent proportional changes in particle inertia
Re	log10(Re)	Represent proportional changes in Reynolds number
Fr	1/Fr	Represent relative gravity strength; Inf becomes zero

```r
#| label: small-St-investigation
#| fig-width: 8
#| fig-height: 8

small_St <- train[train$St <= 0.3, ]

small_St <- small_St[
  order(small_St$Re, small_St$Fr, small_St$St),
]

small_St_table <- small_St[
  c(
    "Re", "Fr", "St",
    "sd_volume", "skewness_volume", "kurtosis_volume"
```

```
  )
]

# Round only the displayed copy
small_St_table[] <- lapply(
  small_St_table,
  function(x) signif(x, 4)
)

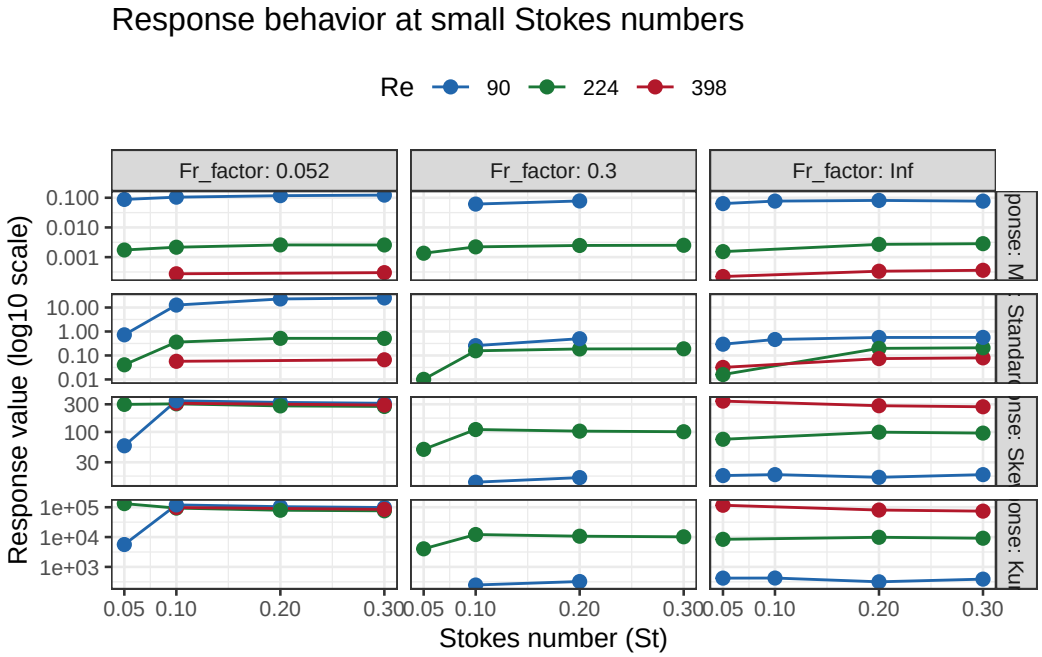
knitr::kable(
  small_St_table,
  row.names = FALSE,
  caption = "Response summaries at small Stokes numbers"
)
```

Table 7: Response summaries at small Stokes numbers

Re	Fr	St	sd_volume	skewness_volume	kurtosis_volume
90	0.052	0.05	0.72580	57.71	5622.0
90	0.052	0.10	12.65000	344.90	119700.0
90	0.052	0.20	22.75000	323.40	104500.0
90	0.052	0.30	25.05000	312.50	97850.0
90	0.300	0.10	0.25710	13.59	247.0
90	0.300	0.20	0.49990	16.31	323.5
90	Inf	0.05	0.29430	17.68	420.3
90	Inf	0.10	0.46390	18.36	423.4
90	Inf	0.20	0.56380	16.50	316.7
90	Inf	0.30	0.56620	18.35	389.2
224	0.052	0.05	0.04075	298.90	132100.0
224	0.052	0.10	0.36100	305.40	93370.0
224	0.052	0.20	0.51800	281.20	79120.0
224	0.052	0.30	0.51520	275.40	75840.0
224	0.300	0.05	0.01006	50.08	4035.0
224	0.300	0.10	0.15640	110.10	12160.0
224	0.300	0.20	0.18660	103.20	10670.0
224	0.300	0.30	0.18940	100.90	10200.0
224	Inf	0.05	0.01594	74.75	8402.0
224	Inf	0.20	0.19750	99.10	9838.0
224	Inf	0.30	0.20880	95.54	9142.0
398	0.052	0.10	0.05705	312.40	97640.0
398	0.052	0.30	0.06594	291.00	84720.0

Re	Fr	St	sd_volume	skewness_volume	kurtosis_volume
398	Inf	0.05	0.03171	340.50	116500.0
398	Inf	0.20	0.07383	283.50	80390.0
398	Inf	0.30	0.07926	272.00	73990.0

```
ggplot(
  response_long[response_long$St <= 0.3, ],
  aes(
    x = St,
    y = Value,
    color = Re_factor,
    group = Re_factor
  )
) +
  geom_line(linewidth = 0.5) +
  geom_point(size = 2) +
  scale_color_manual(values = Re_colors) +
  scale_y_log10() +
  scale_x_continuous(breaks = c(0.05, 0.1, 0.2, 0.3)) +
  facet_grid(
    Response ~ Fr_factor,
    scales = "free_y",
    labeller = label_both
  ) +
  labs(
    title = "Response behavior at small Stokes numbers",
    x = "Stokes number (St)",
    y = "Response value (log10 scale)",
    color = "Re"
  ) +
  theme_bw(base_size = 10) +
  theme(legend.position = "top")
```




```
#| label: compare-St-transformations
#| fig-width: 9
#| fig-height: 5.5

# Two representations of the same observations
St_scale_data <- rbind(
  data.frame(
    response_long,
    X = response_long$St,
    Representation = "St"
  ),
  data.frame(
    response_long,
    X = log10(response_long$St),
    Representation = "log10(St)"
  )
)

St_scale_data$Representation <- factor(
  St_scale_data$Representation,
  levels = c("St", "log10(St)")
)
```

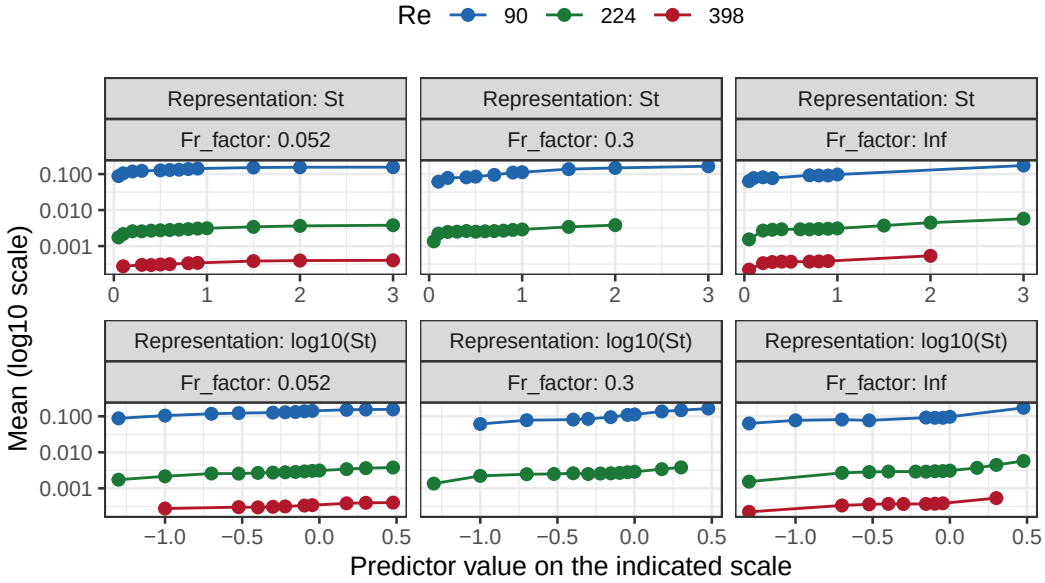
```r
for (response_name in response_labels) {

  plot_data <- St_scale_data[
    St_scale_data$Response == response_name,
  ]

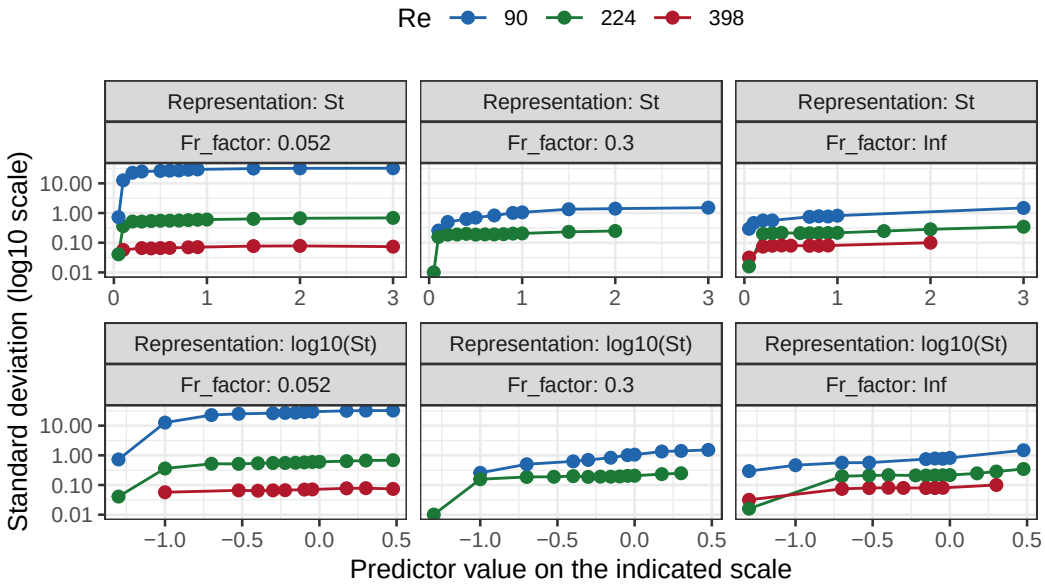
  p <- ggplot(
    plot_data,
    aes(
      x = X,
      y = Value,
      color = Re_factor,
      group = Re_factor
    )
  ) +
    geom_line(linewidth = 0.5) +
    geom_point(size = 1.8) +
    scale_color_manual(values = Re_colors) +
    scale_y_log10() +
    facet_wrap(
      vars(Representation, Fr_factor),
      ncol = 3,
      scales = "free_x",
      labeller = label_both
    ) +
    labs(
      title = paste("Comparing St representations:", response_name),
      x = "Predictor value on the indicated scale",
      y = paste(response_name, "(log10 scale)"),
      color = "Re"
    ) +
    theme_bw(base_size = 10) +
    theme(legend.position = "top")

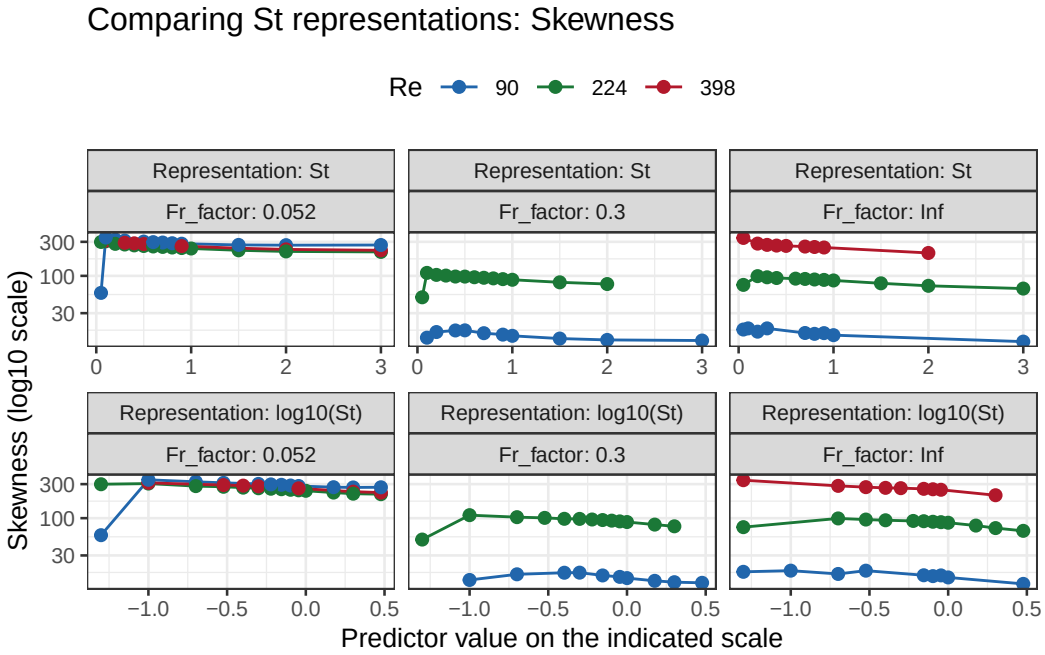
  print(p)
}
```

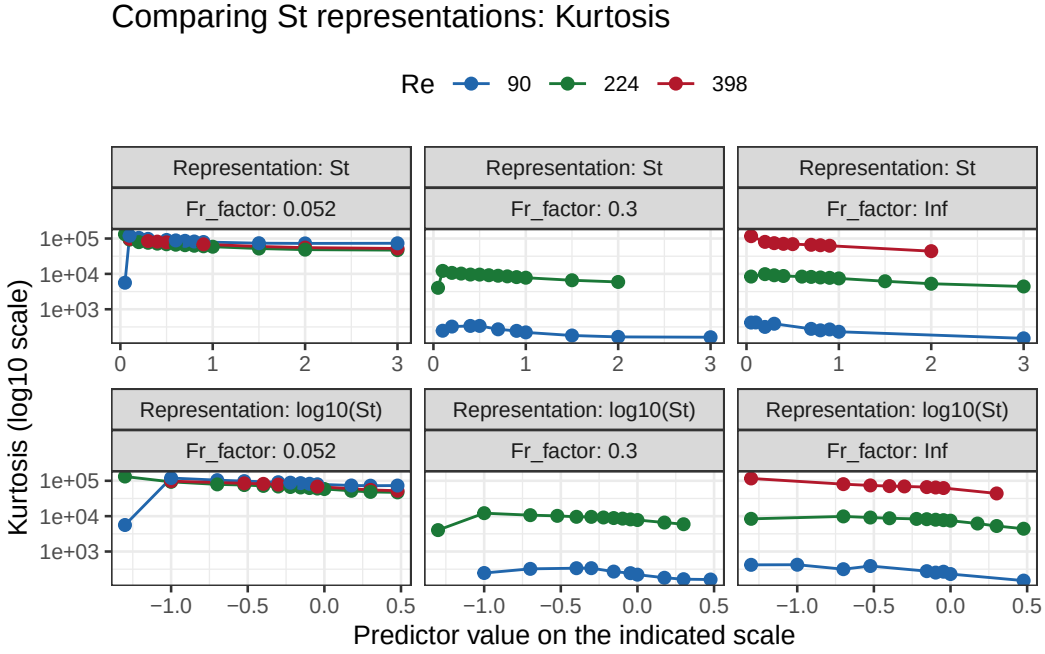
Comparing St representations: Mean



Comparing St representations: Standard deviation








```
#| label: matched-gravity-contrasts
#| fig-width: 8
#| fig-height: 6
```

```r
matching_columns <- c("St", "Re_factor", "Response", "Value")

strong_gravity <- response_long[
  response_long$Fr_factor == "0.052",
  matching_columns
]

no_gravity <- response_long[
  response_long$Fr_factor == "Inf",
  matching_columns
]

# Match on all remaining predictors and the response being compared
gravity_contrasts <- merge(
  strong_gravity,
  no_gravity,
  by = c("St", "Re_factor", "Response"),
  suffixes = c("_strong", "_none")
)

gravity_contrasts$log10_ratio <- log10(
  gravity_contrasts$Value_strong /
    gravity_contrasts$Value_none
)

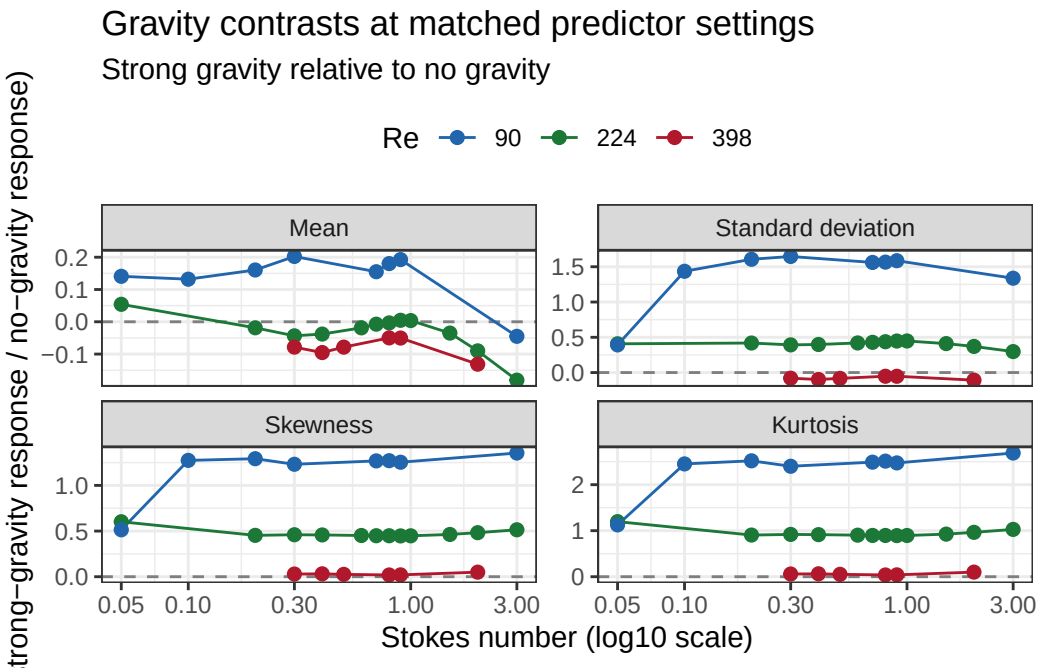
# Show how many matched settings support each Re comparison
matched_counts <- unique(
  gravity_contrasts[c("Re_factor", "St")]
)

knitr::kable(
  as.data.frame(table(Re = matched_counts$Re_factor)),
  caption = "Number of matched St settings for the gravity comparison"
)
```

Table 8: Number of matched St settings for the gravity comparison

Re	Freq
90	8
224	12
398	6

```r
ggplot(
  gravity_contrasts,
  aes(
    x = St,
    y = log10_ratio,
    color = Re_factor,
    group = Re_factor
  )
) +
  geom_hline(
    yintercept = 0,
    linetype = "dashed",
    color = "grey50"
  ) +
  geom_line(linewidth = 0.5) +
  geom_point(size = 2) +
  scale_color_manual(values = Re_colors) +
  scale_x_log10(
    breaks = c(0.05, 0.1, 0.3, 1, 3)
  ) +
  facet_wrap(~ Response, scales = "free_y", ncol = 2) +
  labs(
    title = "Gravity contrasts at matched predictor settings",
    subtitle = "Strong gravity relative to no gravity",
    x = "Stokes number (log10 scale)",
    y = "log10(strong-gravity response / no-gravity response)",
    color = "Re"
  ) +
  theme_bw(base_size = 11) +
  theme(legend.position = "top")
```




```
#| label: create-validation-folds

# Reproducible random assignment
set.seed(325)

K <- 5L

# Stable identifiers matching the current training-data row order
train$row_id <- seq_len(nrow(train))

# Identify the observed simulation regimes
train$regime <- interaction(
  train$Re_factor,
  train$Fr_factor,
  drop = TRUE,
  sep = " / "
)

train$cv_fold <- NA_integer_

# Randomize the labels of the five folds
fold_order <- sample(seq_len(K))
```

```r
# Cycle through fold labels across regimes.
# This keeps total fold sizes and within-regime counts balanced.
offset <- 0L

for (regime_name in levels(train$regime)) {

  rows <- which(train$regime == regime_name)

  assignments <- fold_order[
    ((seq_along(rows) - 1L + offset) %% K) + 1L
  ]

  # Randomly assign these fold labels to observations in this regime
  train$cv_fold[rows] <- assignments[
    sample.int(length(rows))
  ]

  offset <- offset + length(rows)
}

# Save row indices for each training/validation split
cv_folds <- lapply(seq_len(K), function(k) {
  list(
    analysis = which(train$cv_fold != k),
    assessment = which(train$cv_fold == k)
  )
})

names(cv_folds) <- paste0("Fold_", seq_len(K))
```

```r
#| label: check-validation-folds

# Every row must have a valid fold assignment
stopifnot(
  !anyNA(train$cv_fold),
  all(train$cv_fold %in% seq_len(K))
)

# Every observation must appear in exactly one assessment set
assessment_rows <- unlist(
  lapply(cv_folds, function(x) x$assessment),
  use.names = FALSE
```

```
)

stopifnot(
  identical(sort(assessment_rows), seq_len(nrow(train)))
)

# Check separation and regime coverage in every split
for (k in seq_len(K)) {

  analysis_rows <- cv_folds[[k]]$analysis
  assessment_rows <- cv_folds[[k]]$assessment

  stopifnot(
    length(intersect(analysis_rows, assessment_rows)) == 0L,
    length(analysis_rows) + length(assessment_rows) == nrow(train),
    all(
      levels(train$regime) %in%
        as.character(train$regime[analysis_rows])
    ),
    all(
      levels(train$regime) %in%
        as.character(train$regime[assessment_rows])
    )
  )
}

# Overall fold sizes
fold_summary <- data.frame(
  Fold = seq_len(K),
  Training_rows = vapply(
    cv_folds,
    function(x) length(x$analysis),
    integer(1)
  ),
  Validation_rows = vapply(
    cv_folds,
    function(x) length(x$assessment),
    integer(1)
  )
)

knitr::kable(
```

```
  fold_summary,
  row.names = FALSE,
  caption = "Training and validation sample sizes"
)
```

Table 9: Training and validation sample sizes

Fold	Training_rows	Validation_rows
1	71	18
2	72	17
3	71	18
4	71	18
5	71	18

```
# Counts of held-out observations by regime and fold
regime_fold_counts <- table(
  Regime = train$regime,
  Fold = factor(train$cv_fold, levels = seq_len(K))
)

regime_fold_table <- data.frame(
  Regime = rownames(regime_fold_counts),
  as.data.frame.matrix(regime_fold_counts),
  row.names = NULL,
  check.names = FALSE
)

names(regime_fold_table)[-1] <- paste0("Fold ", seq_len(K))

knitr::kable(
  regime_fold_table,
  caption = "Validation observations from each Re / Fr regime"
)
```

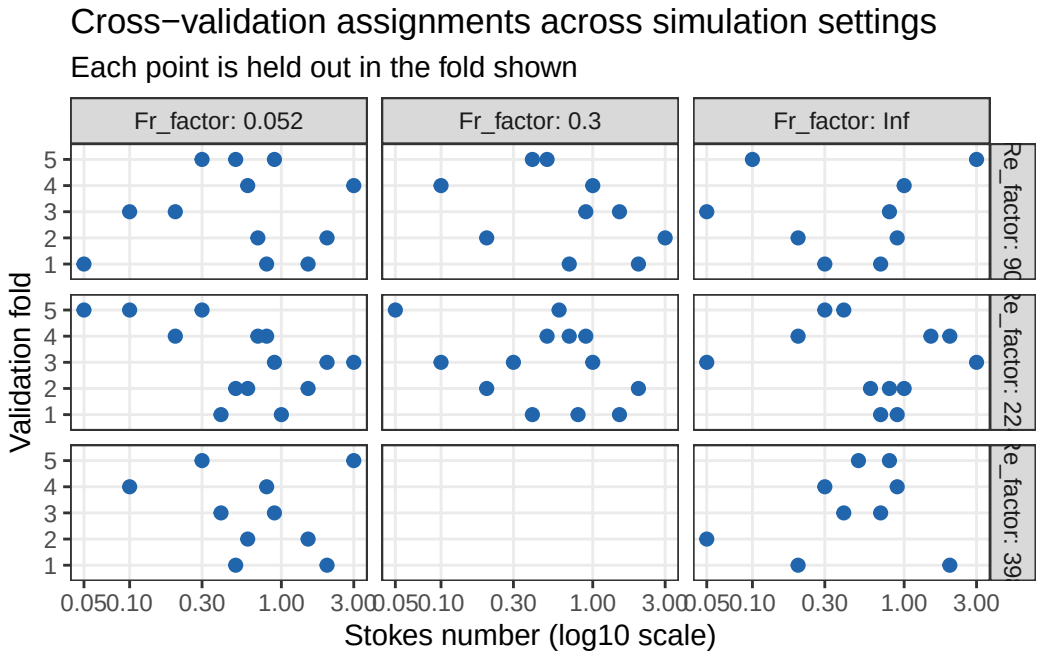
Table 10: Validation observations from each Re / Fr regime

Regime	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5
90 / 0.052	3	2	2	2	3
224 / 0.052	2	3	3	3	3
398 / 0.052	2	2	2	2	2

Regime	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5
90 / 0.3	2	2	2	2	2
224 / 0.3	3	2	3	3	2
90 / Inf	2	2	2	1	2
224 / Inf	2	3	2	3	2
398 / Inf	2	1	2	2	2

```
#| label: validation-design-plot
#| fig-width: 8
#| fig-height: 6

ggplot(
  train,
  aes(
    x = St,
    y = factor(cv_fold, levels = seq_len(K))
  )
) +
  geom_point(
    color = "#2166AC",
    size = 2
  ) +
  scale_x_log10(
    breaks = c(0.05, 0.1, 0.3, 1, 3)
  ) +
  facet_grid(
    Re_factor ~ Fr_factor,
    labeller = label_both,
    drop = FALSE
  ) +
  labs(
    title = "Cross-validation assignments across simulation settings",
    subtitle = "Each point is held out in the fold shown",
    x = "Stokes number (log10 scale)",
    y = "Validation fold"
  ) +
  theme_bw(base_size = 11) +
  theme(panel.grid.minor = element_blank())
```




```
# Check the distribution of small-St observations
small_St_counts <- data.frame(
  Fold = seq_len(K),
  Small_St_validation_rows = vapply(
    cv_folds,
    function(x) {
      as.integer(sum(train$St[x$assessment] <= 0.3))
    },
    integer(1)
  )
)

knitr::kable(
  small_St_counts,
  caption = "Validation observations with St at or below 0.3"
)
```

Table 11: Validation observations with St at or below 0.3

	Fold	Small_St_validation_rows
Fold_1	1	3
Fold_2	2	4

	Fold	Small_St_validation_rows
Fold_3	3	6
Fold_4	4	5
Fold_5	5	8

```
#| label: validation-scoring-functions

# Responses to evaluate separately
cv_responses <- c(
  "mean_volume",
  "sd_volume",
  "skewness_volume",
  "kurtosis_volume"
)

# Score original-scale observed values and held-out predictions
score_predictions <- function(observed, predicted) {

  stopifnot(
    is.numeric(observed),
    is.numeric(predicted),
    length(observed) == length(predicted),
    length(observed) > 0L,
    all(is.finite(observed)),
    all(is.finite(predicted))
  )

  errors <- observed - predicted

  data.frame(
    RMSE = sqrt(mean(errors^2)),
    MAE = mean(abs(errors)),
    Nonpositive_predictions = sum(predicted <= 0)
  )
}

# Retain the validation settings for later modeling steps
cv_settings <- list(
  seed = 325L,
  folds = K,
  responses = cv_responses,
```

```
  primary_metric = "Original-scale RMSE",
  secondary_metric = "Original-scale MAE"
)

cat(
  "Validation setup complete:",
  K, "folds;",
  length(cv_responses), "responses;",
  "primary metric =", cv_settings$primary_metric,
  "\n"
)
```

```
Validation setup complete: 5 folds; 4 responses; primary metric = Original-scale RMSE
```